



Timers

R. Ferrero, P. Bernardi

Politecnico di Torino

Dipartimento di Automatica e Informatica (DAUIN)

Torino - Italy

This work is licensed under the Creative Commons (CC BY-SA) License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>

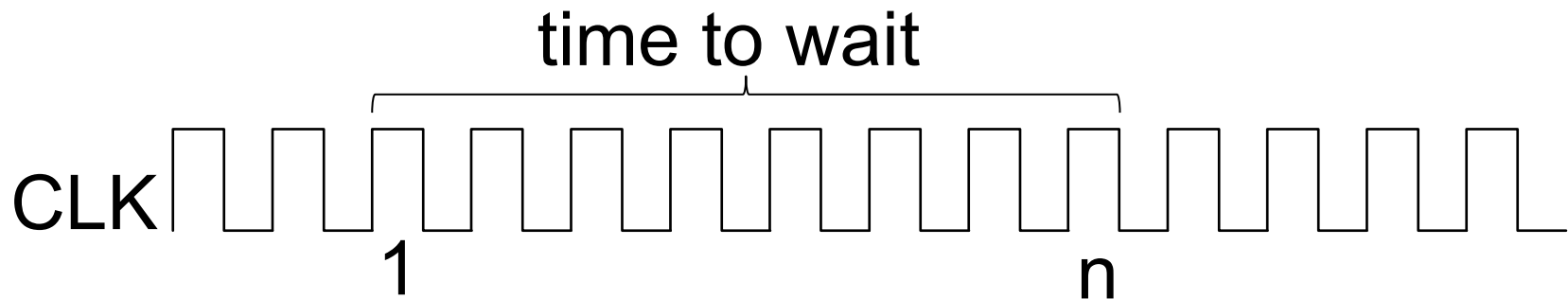


Timers

- Timers are peripherals for synchronizing the system, based on counting
 - to wait for a delay time
 - to perform operations at regular time.
- Usually, when a timer ends counting, the system needs to react.
 - Typically an interrupt handler is entered.
 - While timer counts, the CPU can enter a low-power mode.

Working principle

- Timers are supplied with a (dedicated) clock.
- Timer counting is based on the clock signal.
- Timers include registers to be programmed with the number of clock cycles to count.



COUNT = number of clock cycles

Counting modes

- Two philosophies exist for the timer design:
 - decreasing count: interrupt when count reaches 0
 - increasing count: interrupt when a match value is reached.
- Whatever the counting mode is, the number of clock cycles to count is computed as:
$$\text{count} = \text{time [s]} * \text{frequency [1/s]}$$

Timing computation examples

- How many clock cycles are needed to wait 10 seconds with a frequency of 25 MHz?

$$\begin{aligned}\text{count} &= 10 \text{ [s]} * 25 * 10^6 \text{ [1/s]} = \\ &= 25 * 10^7 = 0x0EE6B280\end{aligned}$$

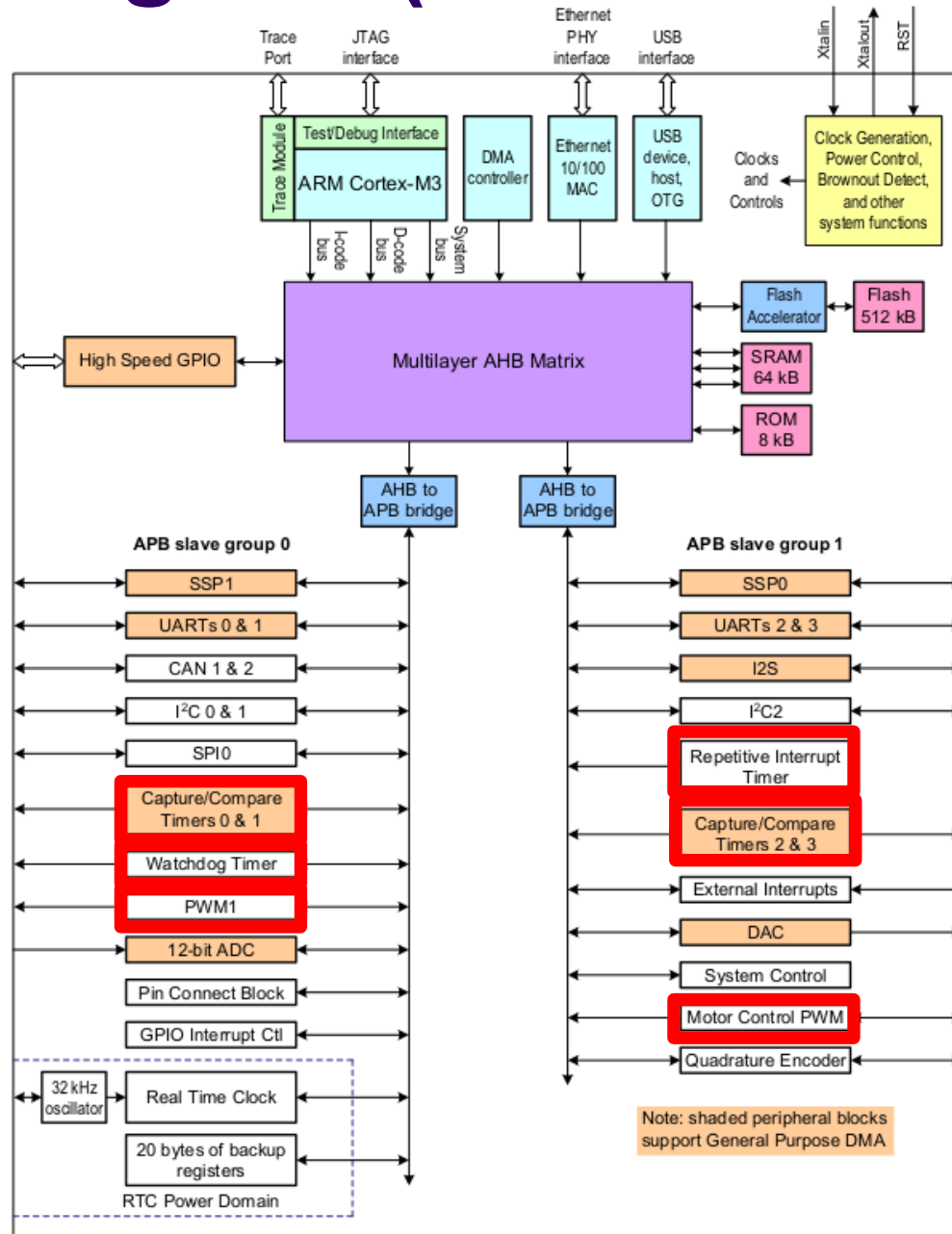
- How many clock cycles are needed to wait 10 milliseconds with a frequency of 100 MHz?

$$\begin{aligned}\text{count} &= 10 * 10^{-3} \text{ [s]} * 100 * 10^6 \text{ [1/s]} = \\ &= 10^6 = \text{count} = 0x000F4240\end{aligned}$$

Timer count limits

- If a timing request is large, the count value could not fit in the timer register.
- Hardware and software features can be used to address this issue
 - HW – Cascade of counters
 - HW – Prescalers
 - SW – Software count of HW events.

Block diagram (user manual p. 10)



Main timers in LPC1768

- 1) System Tick Timer: basic timer with 24-bit counter. It is intended to generate a fixed 10 ms interrupt (with CPU clock set to 100 MHz)
- 2) Capture/Compare timers 0-3: standard timers with 32-bit counter. A prescale counter can change the interrupt rate.
- 3) Repetitive Interrupt Timer: alternative to standard timers with 32-bit counter and a mask register.

Other timers in LPC1768

- PWM: timers for digital signal modulation.
- Motor Control PWM: optimized timer for three-phase AC and DC motor control applications.
- Watchdog timer: it resets the microcontroller within a reasonable amount of time if it enters an erroneous state.

1) System Tick Timer

- A 24-bit timer is used to generate interrupts
 - internal timer: core free-running clock
 - external timer: external reference clock (STCLK)
- Possible uses:
 - alarm timer
 - timing measurement
 - context switch among multiple tasks
- Controlled by four registers.

SYSTICK registers

SYSTICK Register	Size	Address
Control and Status Register	32 bits	0xE000E010
Reload Value Register	24 bits	0xE000E014
Current Value Register	24 bits	0xE000E018
Calibration Value Register	32 bits	0xE000E01C

- SYSTICK Reload Value Register stores the value to reload when timer reaches 0.
- SYSTICK Current Value Register stores the current value of the timer. Writing any number clears its content.

SYSTICK Control and Status Register

Bit	Type	Description
16	R	Read as 1 if counter reaches 0 since last time this register is read; clear to 0 when read or when current counter value (in Current Value Register) is cleared
2	R/W	1 = use processor free running clock 0 = use external reference clock (STCLK)
1	R/W	1 = generate interrupt when timer reaches 0 0 = Do not generate interrupt
0	R/W	1 = enable SYSTICK Timer 0 = disable SYSTICK Timer

SYSTICK Calibration Value Register

Bit	Type	Description
31	R	1 = external reference clock not available 0 = external reference clock available
30	R	1 = calibration value is not exactly 10 ms 0 = calibration value is accurate
23:0	R/W	Calibration value to use for 10 ms timings. Depending on the value of bit 30, this might be exactly 10ms or the closest value. If the calibration value is read as 0, then it is not known. This is probably because the reference clock is an unknown input from the system or scalable dynamically.

SYSTICK configuration

1. Stop timer counter to prevent interrupt triggered accidentally.
 - write 0 to the Control and Status register
2. Set the desired interval between interrupts
 - write a new value to the Reload Value register.
3. Reset the SYSTICK timer counter
 - write any value to the Current Value register
4. Start the SYSTICK timer
 - write proper value to Control and Status register.
 - e.g.: core clock, interrupt enabled, timer enabled

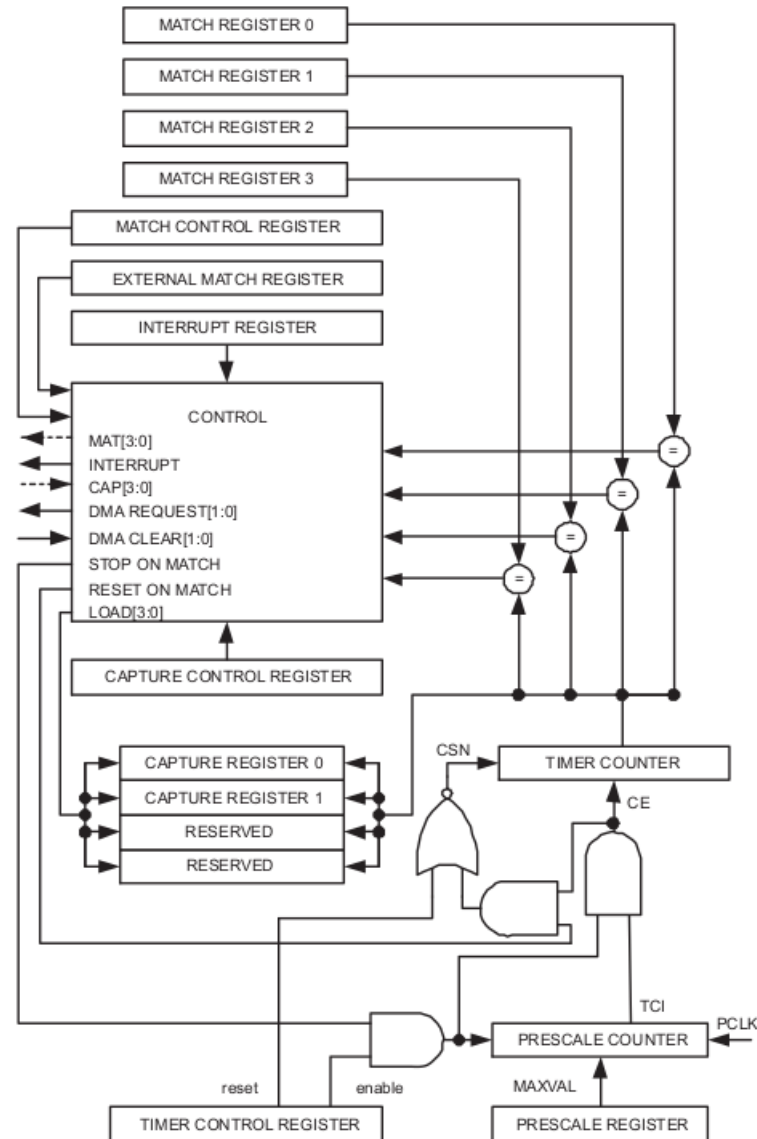
Code for SYSTICK configuration

```
LDR r0, =SYScontrolAndStatusReg
MOV r1, #0
STR r1, [r0] ; step 1
LDR r0, =SYSreloadValueReg
LDR r1, =1023 ; example
STR r1, [r0] ; step 2
LDR r0, =SYScurrentValueReg
STR r1, [r0] ; step 3
LDR r0, =SYScontrolAndStatusReg
MOV r1, #7
STR r1, [r0] ; step 4
```

2) Standard Timers

- The Timer/Counter is designed to count cycles of the peripheral clock (PCLK) or an externally-supplied clock.
- It can optionally generate interrupts or perform other actions at specified timer values, based on four match registers.
- It also includes four capture inputs to trap the timer value when an input signal transitions, optionally generating an interrupt.

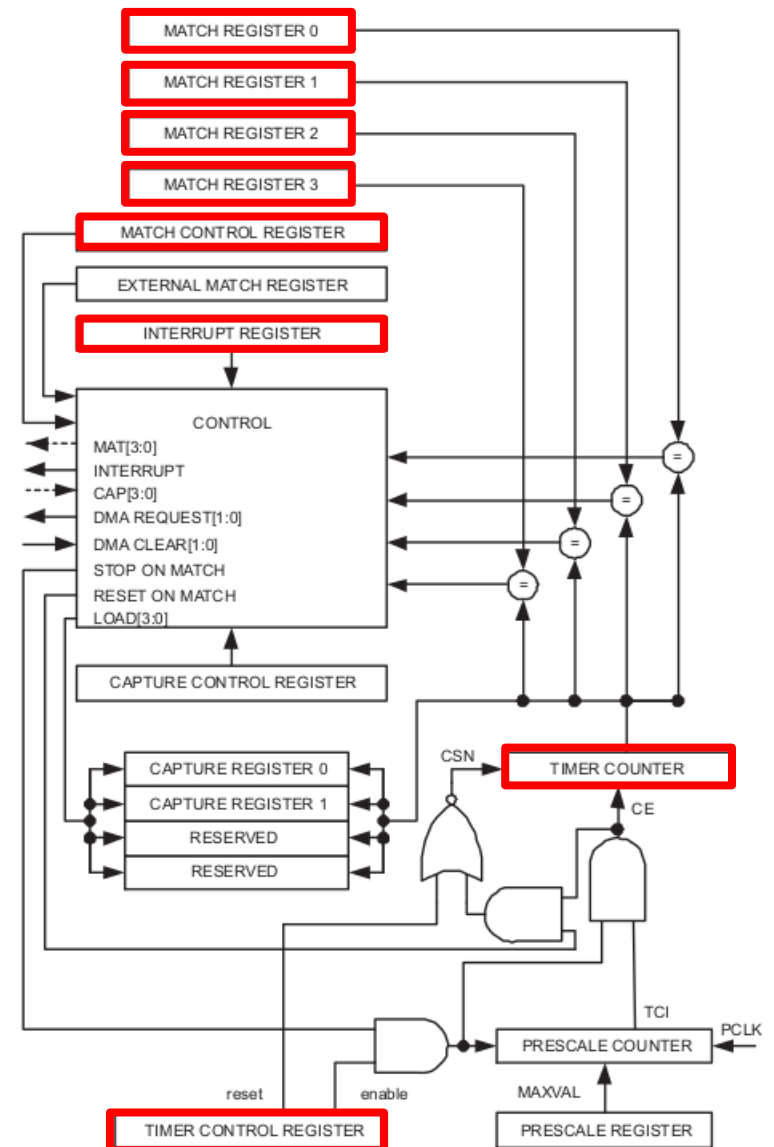
Timer block diagram (page 511)



Main registers (page 503)

Table 426. TIMER/COUNTER0-3 register map

Generic Name	Description	Access
IR	Interrupt Register. The IR can be written to clear interrupts. The IR can be read to identify which of eight possible interrupt sources are pending.	R/W
TCR	Timer Control Register. The TCR is used to control the Timer Counter functions. The Timer Counter can be disabled or reset through the TCR.	R/W
TC	Timer Counter. The 32-bit TC is incremented every PR+1 cycles of PCLK. The TC is controlled through the TCR.	R/W
MCR	Match Control Register. The MCR is used to control if an interrupt is generated and if the TC is reset when a Match occurs.	R/W
MR0	Match Register 0. MR0 can be enabled through the MCR to reset the TC, stop both the TC and PC, and/or generate an interrupt every time MR0 matches the TC.	R/W
MR1	Match Register 1. See MR0 description.	R/W
MR2	Match Register 2. See MR0 description.	R/W
MR3	Match Register 3. See MR0 description.	R/W



Timer counter (TC)

- The Timer Counter counts clock cycles
 - it is incremented when the prescale counter reaches its terminal count.
- It counts up through the value 0xFFFF FFFF and then wrap back to 0x0000 0000.
- When it reaches the upper limit, it does not generate an interrupt
 - a match register should be configured.
- It can be reset before reaching its upper limit.

Timer Control Register (TCR)

The lowest two bits of TCR control the operation of the Timer/Counter:

- Bit 0 = 1 -> enable timer for counting.
- Bit 1 = 1 -> reset counter.

Table 428. Timer Control Register (TCR, TIMERN: TnTCR - addresses 0x4000 4004, 0x4000 8004, 0x4009 0004, 0x4009 4004) bit description

Bit	Symbol	Description	Reset Value
0	Counter Enable	When one, the Timer Counter and Prescale Counter are enabled for counting. When zero, the counters are disabled.	0
1	Counter Reset	When one, the Timer Counter and the Prescale Counter are synchronously reset on the next positive edge of PCLK. The counters remain reset until TCR[1] is returned to zero.	0
31:2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

Match Registers

- The Match register values are continuously compared to the Timer Counter value.
- When the two values are equal, actions can be triggered automatically:
 - to generate an interrupt
 - to reset the Timer Counter
 - to stop the timer
- Actions are controlled by the settings in the Match Control Register.

Match Control Register

- The Match Control Register controls which operations are performed when one of the Match Registers matches the Timer Counter.

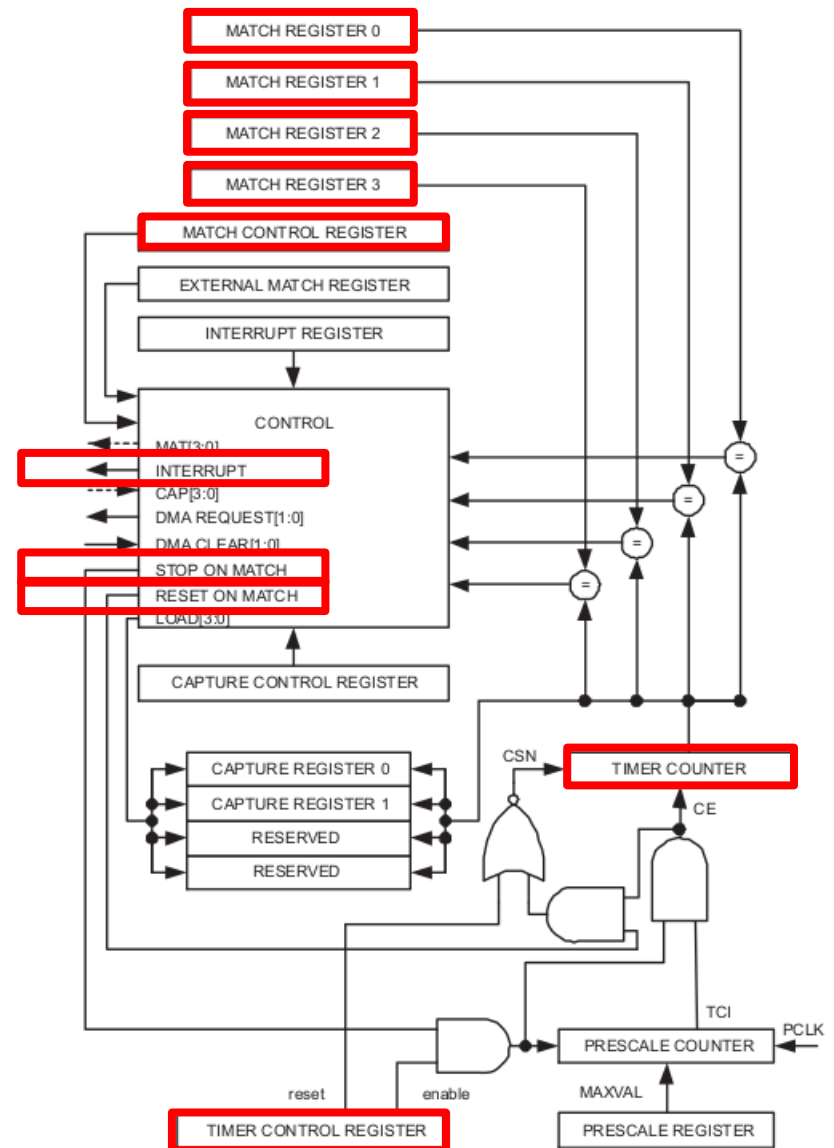
Table 430. Match Control Register (T[0/1/2/3]MCR - addresses 0x4000 4014, 0x4000 8014, 0x4009 0014, 0x4009 4014)
bit description

Bit	Symbol	Value	Description	Reset Value
0	MR0I	1	Interrupt on MR0: an interrupt is generated when MR0 matches the value in the TC.	0
		0	This interrupt is disabled	
1	MR0R	1	Reset on MR0: the TC will be reset if MR0 matches it.	0
		0	Feature disabled.	
2	MR0S	1	Stop on MR0: the TC and PC will be stopped and TCR[0] will be set to 0 if MR0 matches the TC.	0
		0	Feature disabled.	

- Bits 3-5, 6-8, and 9-11 behave in the same way for MR1, MR2, and MR3 respectively.

Three possible behaviors

- Continuous counting with optional interrupt generation on match.
- Stop timer on match with optional interrupt generation.
- Reset timer on match with optional interrupt generation.



Interrupt Register

- The Interrupt Register consists of
 - 4 bits for the match interrupts
 - 2 bits for the capture interrupts.
- If an interrupt is generated, the corresponding bit in the IR is high, otherwise the bit is low.
- Writing 1 to the corresponding IR bit will reset the interrupt, writing 0 has no effect.

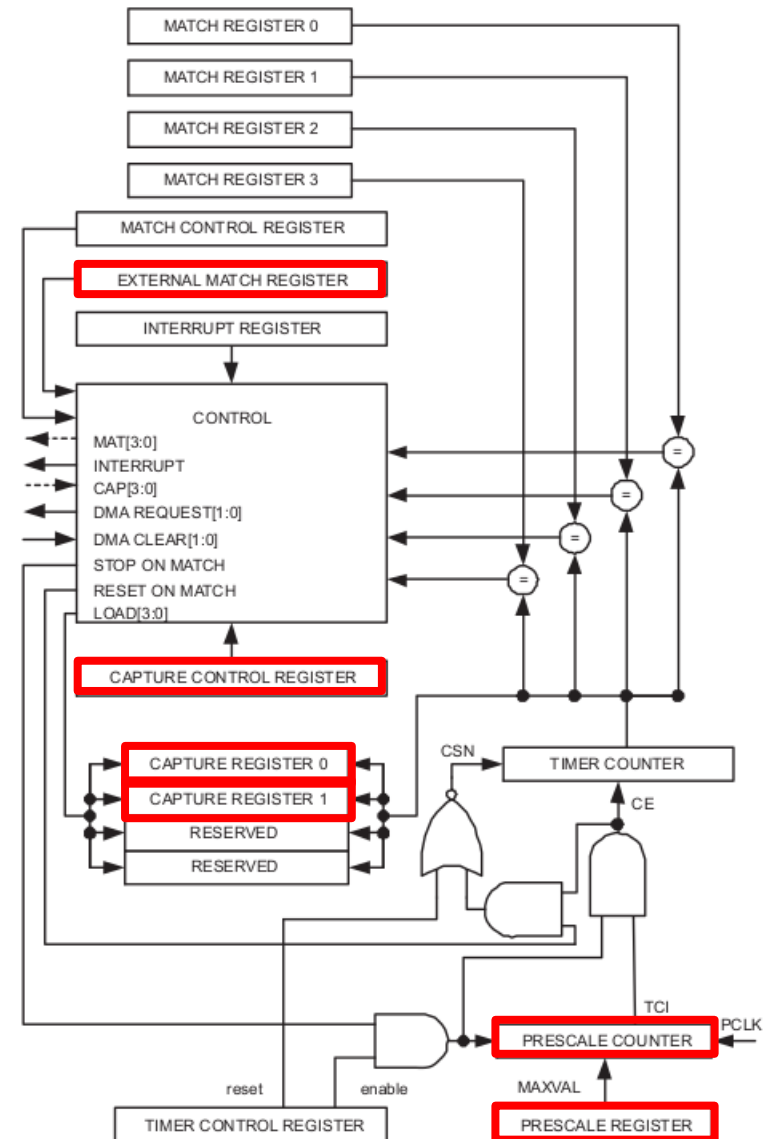
Table 427. Interrupt Register (T[0/1/2/3]IR - addresses 0x4000 4000, 0x4000 8000, 0x4009 0000, 0x4009 4000) bit description

Bit	Symbol	Description	Reset Value
0	MR0 Interrupt	Interrupt flag for match channel 0.	0
1	MR1 Interrupt	Interrupt flag for match channel 1.	0
2	MR2 Interrupt	Interrupt flag for match channel 2.	0
3	MR3 Interrupt	Interrupt flag for match channel 3.	0
4	CR0 Interrupt	Interrupt flag for capture channel 0 event.	0
5	CR1 Interrupt	Interrupt flag for capture channel 1 event.	0
31:6	-	Reserved	-

Other registers (page 503)

Table 426. TIMER/COUNTER0-3 register map

Generic Name	Description	Access
PR	Prescale Register. When the Prescale Counter (below) is equal to this value, the next clock increments the TC and clears the PC.	R/W
PC	Prescale Counter. The 32-bit PC is a counter which is incremented to the value stored in PR. When the value in PR is reached, the TC is incremented and the PC is cleared. The PC is observable and controllable through the bus interface.	R/W
CCR	Capture Control Register. The CCR controls which edges of the capture inputs are used to load the Capture Registers and whether or not an interrupt is generated when a capture takes place.	R/W
CR0	Capture Register 0. CR0 is loaded with the value of TC when there is an event on the CAPn.0(CAP0.0 or CAP1.0 respectively) input.	RO
CR1	Capture Register 1. See CR0 description.	RO
EMR	External Match Register. The EMR controls the external match pins MATn.0-3 (MAT0.0-3 and MAT1.0-3 respectively).	R/W
CTCR	Count Control Register. The CTCR selects between Timer and Counter mode, and in Counter mode selects the signal and edge(s) for counting.	R/W



Registers for prescaling

- The Prescale register stores a 32-bit value
- The Prescale Counter is incremented on every PCLK.
- When the Prescale Counter reaches the value stored in the Prescale register:
 - the Timer Counter is incremented
 - the Prescale Counter is reset on the next PCLK.
- E.g.: if Prescale register is set to 1, the Timer Counter is incremented every 2 PCLKs.

External Match Register

- For every match register, the external match register has
 - an output bit (status of match)
 - a pair of configuration bits (functionality control).
- When a match register equals timer counter, the related bit in the external match register changes according to the configuration bits.

Table 433. External Match Control

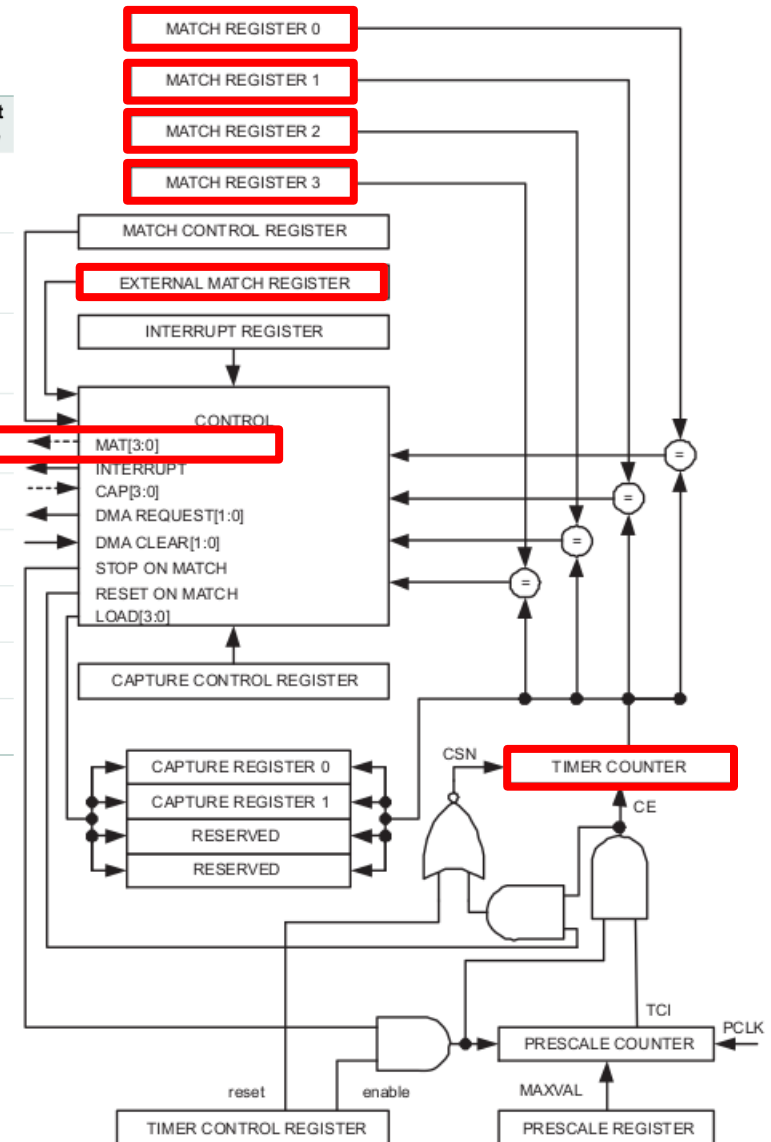
EMR[11:10], EMR[9:8], EMR[7:6], or EMR[5:4]	Function
00	Do Nothing.
01	Clear the corresponding External Match bit/output to 0 (MATn.m pin is LOW if pinned out).
10	Set the corresponding External Match bit/output to 1 (MATn.m pin is HIGH if pinned out).
11	Toggle the corresponding External Match bit/output.

External Match Register

Table 432. External Match Register (T[0/1/2/3]EMR - addresses 0x4000 403C, 0x4000 803C, 0x4009 003C, 0x4009 403C) bit description

Bit	Symbol	Description	Reset Value
0	EM0	External Match 0. When a match occurs between the TC and MR0, this bit can either toggle, go low, go high, or do nothing, depending on bits 5:4 of this register. This bit can be driven onto a MATn.0 pin, in a positive-logic manner (0 = low, 1 = high).	0
1	EM1	External Match 1. When a match occurs between the TC and MR1, this bit can either toggle, go low, go high, or do nothing, depending on bits 7:6 of this register. This bit can be driven onto a MATn.1 pin, in a positive-logic manner (0 = low, 1 = high).	0
2	EM2	External Match 2. When a match occurs between the TC and MR2, this bit can either toggle, go low, go high, or do nothing, depending on bits 9:8 of this register. This bit can be driven onto a MATn.2 pin, in a positive-logic manner (0 = low, 1 = high).	0
3	EM3	External Match 3. When a match occurs between the TC and MR3, this bit can either toggle, go low, go high, or do nothing, depending on bits 11:10 of this register. This bit can be driven onto a MATn.3 pin, in a positive-logic manner (0 = low, 1 = high).	0
5:4	EMC0	External Match Control 0. Determines the functionality of External Match 0. Table 433 shows the encoding of these bits.	00
7:6	EMC1	External Match Control 1. Determines the functionality of External Match 1. Table 433 shows the encoding of these bits.	00
9:8	EMC2	External Match Control 2. Determines the functionality of External Match 2. Table 433 shows the encoding of these bits.	00
11:10	EMC3	External Match Control 3. Determines the functionality of External Match 3. Table 433 shows the encoding of these bits.	00
15:12	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

- The 4 least significant bits of external match register are sent to MAT output.

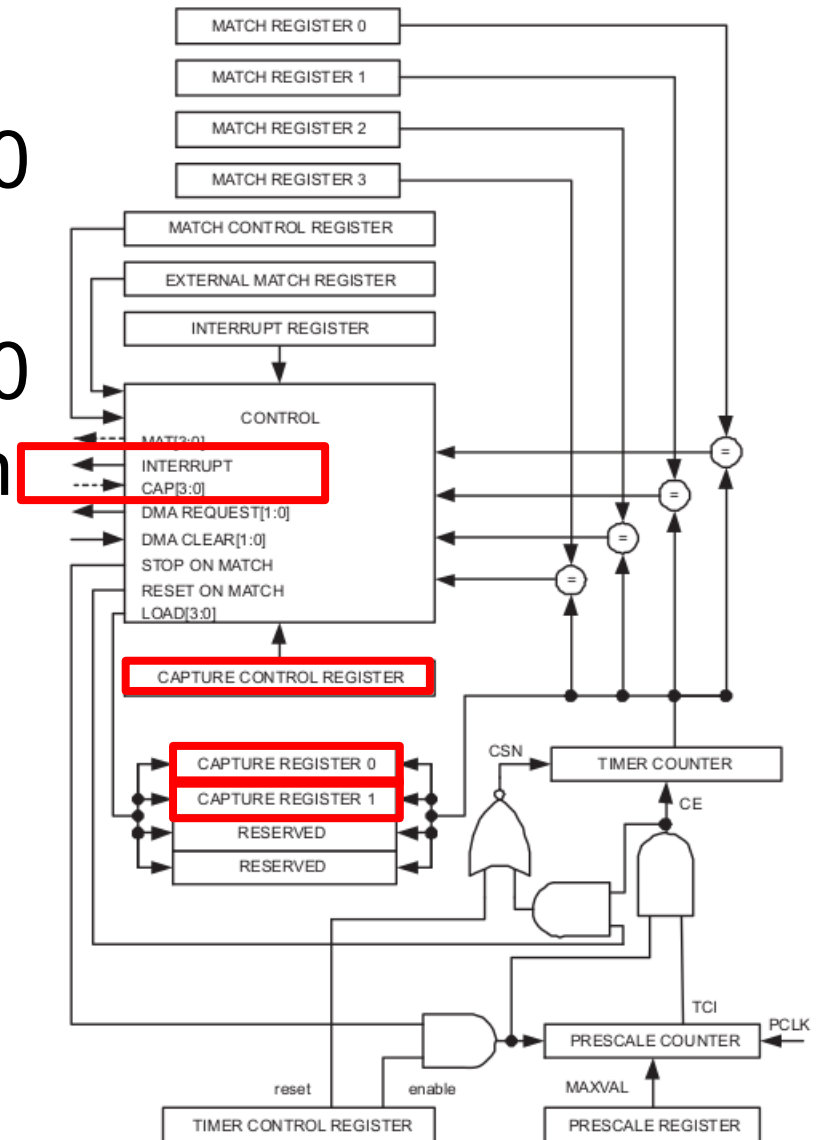


Registers for capturing an event

- 2 Capture Registers: CR0 and CR1.
- 4 Capture Control Registers (one for each pin of CAP input).
- Transition of single CAP pin can be monitored
 - rising edge and/or
 - falling edge
- In such a case, two actions are supported
 - Timer Counter value is stored in a capture register
 - an interrupt is generated.

Bits in Capture Control Register

- Bit 0 = 1: store TC on CR0 on rising edge of CAP pin
- Bit 1 = 1: store TC on CR0 on falling edge of CAP pin
- Bit 2 = 1: generate an interrupt with CR0 load
- Bit 3-5 are the same for CR1.

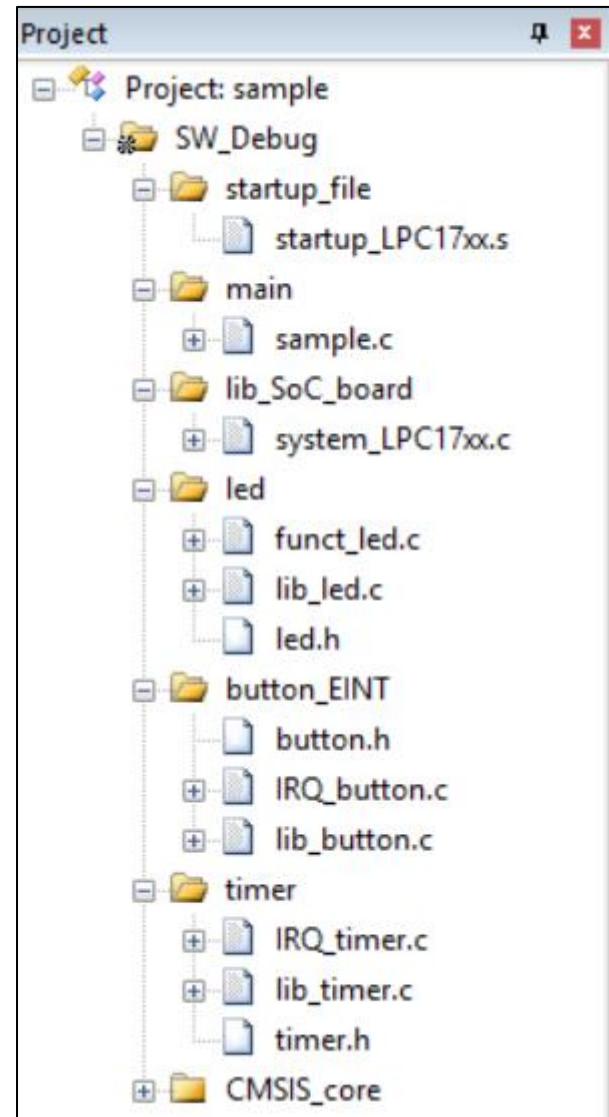


Count Control Register

- It selects between Timer and Counter mode.
- In Timer mode:
 - Prescale Counter is incremented on every PCLK.
 - when the Prescale Counter reaches the value of Prescale register, Timer Counter is incremented.
- In Counter mode:
 - a selected bit of CAP is sampled on every PCLK
 - is the expected transition of CAP bit is recognized (rising and/or falling edge or no change), then the Timer Counter is incremented.

Timer library

- lib_timer.c
 - init_timer(timer_num, timerInterval)
 - enable_timer(timer_num);
 - disable_timer(timer_num);
 - reset_timer(timer_num);
- IRQ_timer.c
 - TIMER0_IRQHandler();
 - TIMER1_IRQHandler();



Delay setup example

- Setup a 1 s delay with 25 MHz frequency
- Raise an interrupt, reset and stop TC

```
/*-----  
    Main Program  
    *-----*/  
int main (void)  
{  
    LED_init();                /* LED Initialization */  
    BUTTON_init();            /* BUTTON Initialization */  
    init_timer(0,0x017D7840);  /* TIMER0 Initialization */  
    enable_timer(0);  
  
    LPC_SC->PCON |= 0x1;      /* power-down mode */  
    LPC_SC->PCON &= 0xFFFFFFFF;  
  
    __ASM("wfi");  
    /* program continues */  
  
}
```

Configuring MCR

1. `init_timer()`
2. GUI
3. configuration wizard.

```
/*-----  
Main Program  
*-----  
int main (void)  
{  
    LED_init();           /* LI  
    BUTTON_init();        /* BU  
    init_timer(0,0x017D7840); /* T  
    enable_timer(0);  
  
    LPC_SC->PCON |= 0x1;    /* power-down mode  
    LPC_SC->PCON &= 0xFFFFFFF;   
  
    __ASM("wfi");  
    /* program continues */  
}
```

Timer 0

Prescaler
PR: 0x00000000
PC: 0x00000000

Timer
TCR: 0x00000000 ☐ Enable
TC: 0x00000000 ☐ Reset

Interrupt Register
IR: 0x00000000

Match Channels
MCR: 0x00000007
MR0: 0x017D7840
MR1: 0x00000000
MR2: 0x00000000
MR3: 0x00000000

☒ Interrupt on MR0
☒ Reset on MR0
☒ Stop on MR0

☐ Interrupt on MR1
☐ Reset on MR1
☐ Stop on MR1

☐ Interrupt on MR2
☐ Reset on MR2
☐ Stop on MR2

☐ Interrupt on MR3
☐ Reset on MR3
☐ Stop on MR3

EMC0: Nothing
EMC1: Nothing
EMC2: Nothing
EMC3: Nothing

☐ External Match 0
☐ External Match 1
☐ External Match 2
☐ External Match 3

☐ MR0 Interrupt
☐ MR1 Interrupt
☐ MR2 Interrupt
☐ MR3 Interrupt

Capture Channels
CCR: 0x00000000
CR0: 0x00000000
CR1: 0x00000000

☐ Rising Edge 0
☐ Falling Edge 0
☐ Interrupt on Event 0
☐ CAP0.0
☐ CR0 Interrupt

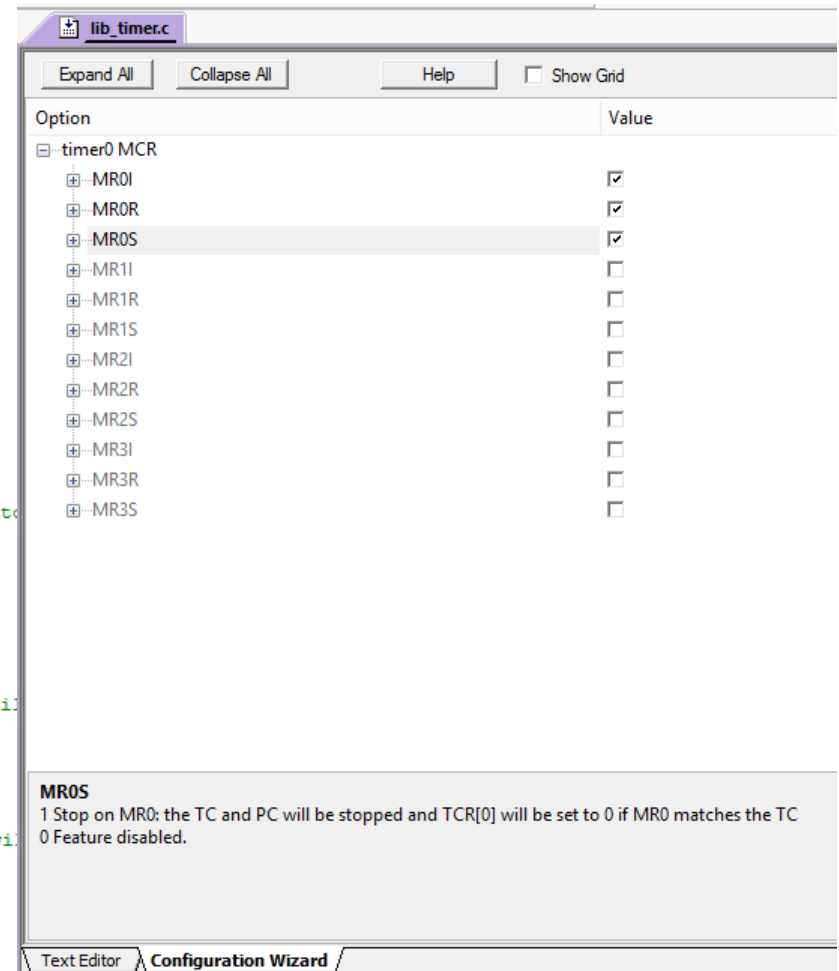
☐ Rising Edge 1
☐ Falling Edge 1
☐ Interrupt on Event 1
☐ CAP0.1
☐ CR1 Interrupt

Count Control
CTCR: 0x00000000
Mode: Timer
Counter Input: CAP0.0

lib_timer.c

- Text editor and configuration wizard

```
91  /*** <<< Use Configuration Wizard in Context Menu >>> ***/
92  // <h> timer0 MCR
93  //   <e.0> MR0I
94  //   <i> 1 Interrupt on MR0: an interrupt is generated when MR0 matches the TC
95  //   <i> 0 This interrupt is disabled
96  //   </e>
97  //   <e.1> MR0R
98  //   <i> 1 Reset on MR0: the TC will be reset if MR0 matches it.
99  //   <i> 0 Feature disabled.
100 //   </e>
101 //   <e.2> MR0S
102 //   <i> 1 Stop on MR0: the TC and PC will be stopped and TCR[0] will be set to 0 if MR0 matches the TC
103 //   <i> 0 Feature disabled.
104 //   -----
105 //   <i> 0 Feature disabled.
106 //   </e>
107 //   <e.11> MR3S
108 //   <i> 1 Stop on MR3: the TC and PC will be stopped and TCR[3] will be set to 0 if MR3 matches the TC
109 //   <i> 0 Feature disabled.
110 //   </e>
111 LPC_TIMER0->MCR = 7;
112 // </h>
113 /*** <<< end of configuration section >>> ***/
```



Enable_timer function

- The timer is started.

```
/*-----  
Main Program  
*-----  
int main (void)  
{  
    LED_init();  
    BUTTON_init();  
    init_timer(0,0x017D7840);  
    enable_timer(0);  
  
    LPC_SC->PCON |= 0x1;    /* power-down m  
    LPC_SC->PCON &= 0xFFFFFFF0;  
  
    __ASM("wfi");  
    /* program continues */  
}
```

Timer 0

Prescaler
PR: 0x00000000
PC: 0x00000000

Timer
TCR: 0x00000001
TC: 0x00052A38

☒ Enable
☐ Reset

Interrupt Register
IR: 0x00000000

Match Channels
MCR: 0x00000007
MR0: 0x017D7840
MR1: 0x00000000
MR2: 0x00000000
MR3: 0x00000000
EMR: 0x00000000

☒ Interrupt on MR0
☒ Reset on MR0
☒ Stop on MR0

☐ Interrupt on MR1
☐ Reset on MR1
☐ Stop on MR1

☐ Interrupt on MR2
☐ Reset on MR2
☐ Stop on MR2

☐ Interrupt on MR3
☐ Reset on MR3
☐ Stop on MR3

EMC0: Nothing
EMC1: Nothing
EMC2: Nothing
EMC3: Nothing

☐ External Match 0
☐ External Match 1
☐ External Match 2
☐ External Match 3

☐ MR0 Interrupt
☐ MR1 Interrupt
☐ MR2 Interrupt
☐ MR3 Interrupt

Capture Channels
CCR: 0x00000000
CR0: 0x00000000
CR1: 0x00000000

☐ Rising Edge 0
☐ Falling Edge 0
☐ Interrupt on Event 0
☐ CAP0.0
☐ CR0 Interrupt

☐ Rising Edge 1
☐ Falling Edge 1
☐ Interrupt on Event 1
☐ CAP0.1
☐ CR1 Interrupt

Count Control
CTCR: 0x00000000
Mode: Timer
Counter Input: CAP0.0

Timing check

- Measure time between breakpoints.

```
15  /*-----*/
16  Main Program
17  *-----*/
18  int main (void)
19  {
20      LED_init();          /* LED Initialization */
21      BUTTON_init();       /* BUTTON Initialization */
22      init_timer(0,0x017D7840); /* TIMER0 Initialization */
23      enable_timer(0);
24
25      LPC_SC->PCON |= 0x1; /* power-down mode */
26      LPC_SC->PCON &= 0xFFFFFFFF;
27
28      __ASM("wfi");
29      /* program continues */
30
31  }
```

```
10
11  void TIMER0_IRQHandler (void)
12  {
13      LPC_TIM0->IR = 1;    /* clear interrupt flag */
14      return;
15  }
16
17  void TIMER1_IRQHandler (void)
18  {
19      LPC_TIM1->IR = 1;    /* clear interrupt flag */
20      return;
21  }
```

Initial time

```
3  *
4  * This software is supplied "AS IS" without
5  *
6  * Copyright (c) 2019 Politecnico di Torino
7  *
8
9  #include <stdio.h>
10 #include "LPC17xx.h" /*
11 #include "led/led.h"
12 #include "button/button.h"
13 #include "timer/timer.h"
14
15 /*-----
16 Main Program
17 *-----
18 int main (void)
19 {
20     LED_init(); /*
21     BUTTON_init(); /*
22     init_timer(0, 0x017D7840); /*
23     enable_timer(0);
24
25     LPC_SC->PCON |= 0x1; /* power-down mod
26     LPC_SC->PCON &= 0xFFFFFFFED;
27
28     __ASM("wfi");
29     /* program continues */
30
31 }
32
33
```

Timer 0

Prescaler
PR: 0x00000000
PC: 0x00000000

Timer
TCR: 0x00000000 ☐ Enable
TC: 0x00000000 ☐ Reset

Interrupt Register
IR: 0x00000000

Match Channels

MCR: 0x00000003 EMR: 0x00000000

MR0: 0x017D7840 MR1: 0x00000000 MR2: 0x00000000 MR3: 0x00000000

☒ Interrupt on MR0 ☐ Interrupt on MR1 ☐ Interrupt on MR2 ☐ Interrupt on MR3
☒ Reset on MR0 ☐ Reset on MR1 ☐ Reset on MR2 ☐ Reset on MR3
☐ Stop on MR0 ☐ Stop on MR1 ☐ Stop on MR2 ☐ Stop on MR3

EMC0: Nothing EMC1: Nothing EMC2: Nothing EMC3: Nothing

☐ External Match 0 ☐ External Match 1 ☐ External Match 2 ☐ External Match 3
☐ MR0 Interrupt ☐ MR1 Interrupt ☐ MR2 Interrupt ☐ MR3 Interrupt

Capture Channels

CCR: 0x00000000

CR0: 0x00000000 CR1: 0x00000000

☐ Rising Edge 0 ☐ Rising Edge 1
☐ Falling Edge 0 ☐ Falling Edge 1
☐ Interrupt on Event 0 ☐ Interrupt on Event 1
☐ CAP0.0 ☐ CAP0.1
☐ CR0 Interrupt ☐ CR1 Interrupt

Count Control

CTCR: 0x00000000 Mode: Timer Counter Input: CAP0.0

Final time

```
1  /*****-----File Info-----*****/
2  **-----File Info-----**
3  ** File name:          IRQ_timer.c
4  ** Descriptions:      interrupt handlers
5  ** Correlated files:  timer.h
6  **-----File Info-----**
7  *****/
8  #include "lpc17xx.h"
9  #include "timer.h"
10
11 void TIMERO_IRQHandler (void)
12 {
13     LPC_TIM0->IR = 1; /* clear interrupt
14     return;
15 }
16
17 void TIMER1_IRQHandler (void)
18 {
19     LPC_TIM1->IR = 1; /* clear interrupt
20     return;
21 }
22
23
```

Clear
Interrupt flag

Timer 0

Prescaler: PR: 0x00000000 PC: 0x00000000

Timer: TCR: 0x00000001 TC: 0x00000002

☒ Enable ☐ Reset

Interrupt Register: IR: 0x00000001

Match Channels

MCR: 0x00000003 EMR: 0x00000000

MR0: 0x017D7840 MR1: 0x00000000 MR2: 0x00000000

☒ Interrupt on MR0 ☐ Interrupt on MR1 ☐ Interrupt on MR2 ☐ Interrupt on MR3

☒ Reset on MR0 ☐ Reset on MR1 ☐ Reset on MR2 ☐ Reset on MR3

☐ Stop on MR0 ☐ Stop on MR1 ☐ Stop on MR2 ☐ Stop on MR3

EMC0: Nothing EMC1: Nothing EMC2: Nothing EMC3: Nothing

☐ External Match 0 ☐ External Match 1 ☐ External Match 2 ☐ External Match 3

☒ MR0 Interrupt ☐ MR1 Interrupt ☐ MR2 Interrupt ☐ MR3 Interrupt

Capture Channels

CCR: 0x00000000

CR0: 0x00000000 CR1: 0x00000000

☐ Rising Edge 0 ☐ Rising Edge 1

☐ Falling Edge 0 ☐ Falling Edge 1

☐ Interrupt on Event 0 ☐ Interrupt on Event 1

☐ CAP0.0 ☐ CAP0.1

☐ CR0 Interrupt ☐ CR1 Interrupt

Count Control

CTCR: 0x00000000 Mode: Timer Counter Input: CAP0.0

Interrupt
register

Exercises

- Setup regular interval time interruptions; in the meanwhile the system enters in a low-power state.
- Raise another interrupt with a different period (using another match register).
- Figure out the maximum delay that can be obtained when using 25 MHz. Try to overcome this limitation using
 - HW resources available in the peripheral
 - SW methods.

3) Repetitive Interrupt Timer (RIT)

- It has four registers:
 - RICOUNTER: 32-bit counter running from PCLK. Counter can be free-running, or be reset by a generated interrupt.
 - RICOMPVAL: 32-bit compare value.
 - RIMASK: 32-bit compare mask. An interrupt is generated when the counter value equals the compare value, after masking. This allows for combinations not possible with a simple compare.
 - RICTRL: control register.

RI Mask register

- The Mask register holds the 32-bit mask value.
- A one written to any bit overrides the result of the comparison for the corresponding bit of the counter and compare register
 - it causes the comparison of the register bits to be always true.

RI Control register

Table 437. RI Control register (RCTRL - address 0x400B 0008) bit description

Bit	Symbol	Value	Description	Reset value
0	RITINT		Interrupt flag	0
		1	This bit is set to 1 by hardware whenever the counter value equals the masked compare value specified by the contents of RICOMPVAL and RIMASK registers. Writing a 1 to this bit will clear it to 0. Writing a 0 has no effect.	
		0	The counter value does not equal the masked compare value.	
1	RITENCLR		Timer enable clear	0
		1	The timer will be cleared to 0 whenever the counter value equals the masked compare value specified by the contents of RICOMPVAL and RIMASK registers. This will occur on the same clock that sets the interrupt flag.	
		0	The timer will not be cleared to 0.	
2	RITENBR		Timer enable for debug	1
		1	The timer is halted when the processor is halted for debugging.	
		0	Debug has no effect on the timer operation.	
3	RITEN		Timer enable.	1
		1	Timer enabled. Remark: This can be overruled by a debug halt if enabled in bit 2.	
		0	Timer disabled.	
31:4	-	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA